Ottermap Technical Summary

Turf/Grass Semantic Segmentation from Aerial Imagery — Open Vision/ML Engineer Intern Challenge

Development Environment Disclosure

This submission was assembled in a sandboxed environment with no GPU, no internet access, and none of **torch, transformers, rasterio, geopandas, shapely** installed. Rather than write untested code, every part of the pipeline that could be validated without those libraries — georeferencing math, AOI-aware tiling, mask rasterization, morphological cleaning, mask-to-polygon-to-GeoJSON conversion, and sliding-window stitching — was reimplemented with available substitutes (cv2/numpy/PIL) and run against the **real provided imagery and labels** to prove correctness before being finalized in the production rasterio/geopandas code. Model training was not executed (no GPU/internet); src/train.py and inference.py are complete and ready to run in a standard GPU environment. All quantitative claims below are either (a) measured directly against the provided data in this session, or (b) explicitly marked as expected/pending training.

1. Approach

Model Architecture

SegFormer-B2 (nvidia/segformer-b2-finetuned-ade-512-512, HuggingFace), fine-tuned as a binary (turf vs. background) segmenter, replacing the 150-class ADE20K head. SegFormer's hierarchical MiT transformer encoder + lightweight all-MLP decoder was chosen over CNN alternatives (U-Net++, DeepLabV3+) for its strong ImageNet/ADE20K-pretrained features and good transfer to aerial imagery reported in prior remote-sensing literature, without the very large parameter count of B5 (~84M params).

B2 instead of B5: with only 3 source images (~200 AOI-filtered 512×512 tiles after tiling — see Section 2), fine-tuning an 84M-parameter transformer risks overfitting and costs materially more compute within a single-GPU, 72-hour budget. B2 (~25M params) keeps the same architecture family and pretrained features, trains faster, and is more stable in low-data regimes. Backbone is a one-flag config change (--backbone b0|b2|b5) for teams with more data or compute.

Training Methodology

- **Two-phase fine-tuning:** encoder frozen for the first 10 epochs (decoder-only warmup on random-init binary head), then encoder unfrozen with LR reduced 10x, for the remainder of training.
- **Loss:** AOI-masked Dice + weighted cross-entropy + IoU combo loss (src/losses.py). Loss and metrics are computed *only inside the annotated AOI* — see Section 2 for why this is not optional.
- **Evaluation protocol — Leave-One-Location-Out (LOLO):** with only 3 source images from 3 distinct geographies, a random tile-level train/val split would leak near-duplicate rooftops/lawns/lighting between splits and overstate generalization. Instead, train.py --mode lolo runs 3 training loops, each holding out one full source image as validation, and reports the mean/std IoU across the 3 holdouts as the generalization estimate. The deployment model (--mode final) is then retrained on all 3 images.
- Mixed-precision (AMP) training and inference; AdamW, weight decay 1e-4, batch size 8, up to 40 epochs with best-checkpoint selection on held-out IoU.

2. Dataset Preparation

Data Findings That Shaped the Pipeline

Direct inspection of the provided files (not assumption) surfaced two facts that changed the design:

- **GCP-based georeferencing:** all 3 GeoTIFFs carry 4 corner tiepoints (ModelTiepointTag ×4) rather than a single tiepoint + pixel scale. GDAL/rasterio expose this as Ground Control Points — naively reading src.transform returns an identity matrix. src/geo_utils.py routes through rasterio.transform.from_gcps(). Verified: a closed-form affine derived from the 4 real corner tiepoints reproduces all 12 corner coordinates (3 images × 4 corners) to floating-point exactness (tests/test_geo_math.py), and implied ground-sample-distance is a plausible 0.1–0.3 m/px for all 3 images.
- **The two feature-layer folders mean different things.** GeoJSON/*.geojson holds the actual turf/grass polygon labels (95 / 50 / 16 polygons for images 1/2/3; id and Area properties are unpopulated — single-class dataset). ShapeFile/*.geojson is *not* more labels — each file is one bare geometry object matching the image extent: the **Area of Interest** that was exhaustively annotated. Measured AOI coverage was only 28.4% / 73.4% / 60.5% of the three images respectively —

meaning the majority of image 1, in particular, was never labelled. Treating unlabelled pixels as confident negatives would actively teach the model to under-predict turf. Every tile, loss computation, and metric in this repo is restricted to AOI pixels (dataset_prep.py, losses.py, metrics.py).

Preprocessing

GeoTIFFs → RGB array + rasterized grass mask + rasterized AOI validity mask, all at native resolution, then AOI-aware sliding-window tiling (512×512, 64px overlap, tiles dropped below 30% AOI coverage). Verified against the real data: **197 usable tiles** across the 3 images (22 / 64 / 111), reflecting each image's AOI coverage and grass fraction (9.1% / 46.4% / 43.4% grass pixels respectively).

Augmentation

With so few source images, augmentation carries real weight for generalization. Pipeline (Albumentations, src/dataset.py): horizontal/vertical flip, 90° rotation, shift/scale/rotate (±15°, ±15% scale — approximates GSD variation across sensors), one-of {brightness/contrast, CLAHE, HSV jitter} for illumination/seasonal variation, one-of {blur, Gaussian noise, JPEG compression artifacts} for sensor/compression variation, and coarse dropout. Aerial imagery has no canonical "up," so flips/rotations are safe (also exploited in test-time augmentation at inference — 4-way flip/rotate averaging, src/tiling.py).

3. GIS Output Pipeline — Verified

binary mask → morphological open+close (speckle removal / small-gap fill) → rasterio.features.shapes → Shapely polygons → simplification → GeoPandas GeoDataFrame with area_sqm/area_acres → GeoJSON + Shapefile. Verified end-to-end against real data (cv2-contour equivalent, since rasterio is unavailable in this sandbox): injected 2,000 speckle pixels into a real grass mask; morphological cleaning reduced disagreement with the clean mask by 89% (2000 → 212 px). Running the full mask→polygon→GeoJSON round trip on image 2's real grass mask recovered **49 polygons vs. 50 in the original label set** (one pair merged under closing — expected and visually confirmed correct), with a plausible total turf area of ~18 acres for that school-campus scene. Sliding-window stitching (src/tiling.py) was verified to produce seam-free output at tile-stride boundaries on a synthetic smooth signal (mean adjacent-pixel discontinuity < 0.05 on a [0,1] probability scale).

4. Results

Pipeline correctness (measured this session): georeferencing exact-corner match (12/12 corners), 197/≈260 candidate tiles retained after AOI filtering, 89% speckle-noise reduction from morphological cleaning, 49/50 polygon recovery on round-trip, zero seam artifacts in stitched inference. 10 automated tests pass (tests/test_*.py, runnable without any deep-learning or geo library).

Model accuracy — pending: no GPU/internet was available to run src/train.py in this session, so IoU/Dice/F1 on held-out imagery are not yet measured. Running python src/train.py --mode lolo in a GPU environment will populate models/lolo_results.json with per-location and mean/std IoU; this is the number to report as the generalization estimate, since each holdout is a full unseen location, not a random tile split.

Expected failure modes (to watch for once trained, based on the data itself): image 1's low AOI coverage (28%) and low grass fraction (9%) gives it far fewer positive examples than images 2–3, so a model trained with image 1 held out (LOLO) is expected to show the lowest IoU of the three holdout runs. Sports turf (image 3, baseball outfields) has a visually distinct uniform mowed texture vs. residential lawns (image 1) and campus lawns (image 2) — cross-holdout performance on image 3 is a reasonable proxy for how well the model will generalize to sports-facility imagery specifically, vs. general residential/campus turf.

5. Improvements With More Time/Data

- Run the LOLO protocol and report real numbers; use it to pick threshold/backbone rather than defaults.
- Source several more AOI+label sets (ideally same annotation methodology) to reduce reliance on augmentation and allow a genuine random-split validation set in addition to LOLO.
- Add a lightweight classical baseline (e.g. NDVI-like vegetation index + Otsu threshold, or a random-forest on color/texture features) purely as a sanity floor — if SegFormer doesn't clearly beat it given the small training set, that's important to know.

- Try SAM or Grounding-DINO + SAM as a zero-shot/few-shot comparison point, particularly since turf boundaries are geometrically simple (large, smooth regions) — a promptable segmenter may need little to no fine-tuning.
- ONNX export for lighter-weight/faster deployment inference once a checkpoint exists.
- Multi-band (NIR) input if 4-band NAIP-style imagery is available — vegetation indices (NDVI) are a strong signal for turf vs. impervious surface that pure RGB doesn't capture as cleanly.